



SQL

Write Better
Queries.
Get Better
Results.

50 SQL QUERIES EVERY DEVELOPER

Must Know

From Beginner to Advanced

- ✓ Real Examples
- 👥 Interview Friendly
- ★ Best Practices
- 💡 Pro Tips
- ✗ Common Mistakes



SQL is the language behind every powerful application and data-driven decision.



1

BASIC QUERIES – SELECT

The **SELECT** statement is used to retrieve data from a database.



WHAT IT DOES?

Retrieves data from one or more columns in a table.



WHY USE IT?

SELECT is the most basic and most used command in SQL.

It helps you fetch the data you need from a table.



PRO TIP

Avoid using SELECT * in production.

Always fetch only the columns you actually need.

It improves performance and reduces unnecessary data transfer.



EXAMPLE

- Get all columns from employees

```
SELECT *  
FROM employees;
```

- Get specific columns

```
SELECT name, salary, department  
FROM employees;
```



SAMPLE TABLE: employees

id	name	salary	department
1	Riya	60000	HR
2	Aman	75000	IT
3	Karan	50000	Finance



INTERVIEW QUESTION

What is the difference between SELECT * and SELECT column1, column2?



EXPECTED ANSWER:

SELECT * fetches all columns, which can be slow and consume more memory. Specific columns are faster and secure.



2

BASIC QUERIES – WHERE



The **WHERE** clause is used to filter records that meet a specific condition.



WHAT IT DOES?

Filters rows based on a specified condition.



WHY USE IT?

Helps you fetch only the required data instead of retrieving all rows.



PRO TIP

Use WHERE clause early in your query to reduce the number of rows processed and improve performance.



EXAMPLE

- Employees with salary greater than 60000

```
SELECT *  
FROM employees  
WHERE salary > 60000;
```

- Employees in IT department

```
SELECT *  
FROM employees  
WHERE department = 'IT';
```

SAMPLE TABLE: employees

id	name	salary	department
1	Riya	60000	HR
2	Aman	75000	IT
3	Karan	50000	Finance
4	Neha	82000	IT



INTERVIEW QUESTION

How does WHERE clause improve query performance?



EXPECTED ANSWER:

It filters data before processing, which reduces the amount of data the database needs to read and process.



3

BASIC QUERIES – DISTINCT



The **DISTINCT** keyword is used to return only unique (different) values.



WHAT IT DOES?

Removes duplicate rows and returns only unique values.



WHY USE IT?

Useful when you want a list of unique values from a column.



PRO TIP

DISTINCT works on all selected columns together. If any column value is different, the row is considered unique.



EXAMPLE

- Get unique departments

```
SELECT DISTINCT department
FROM employees;
```

Result

department
HR
IT
Finance

4

BASIC QUERIES – ORDER BY & LIMIT



ORDER BY sorts the result. **LIMIT** restricts the number of rows returned.



WHAT IT DOES?

ORDER BY sorts the result. **LIMIT** restricts the number of rows returned.



WHY USE IT?

Helps in displaying data in an order and controlling the number of results.



PRO TIP

Always use **ORDER BY** with **LIMIT** to ensure consistent results.



EXAMPLE

- Order by salary (High to Low)

```
SELECT * FROM employees
ORDER BY salary DESC;
```

- Get top 2 highest salary

```
SELECT * FROM employees
ORDER BY salary DESC
LIMIT 2;
```



5

AGGREGATE FUNCTIONS



Aggregate functions perform calculations on a set of values and return a single value.

01

COUNT()

Counts the number of rows that match a condition.

EXAMPLE

```
SELECT COUNT(*) AS total_employees
FROM employees;
```

SAMPLE RESULT

total_employees
4

02

SUM()

Returns the total sum of a numeric column.

EXAMPLE

```
SELECT SUM(salary) AS total_salary
FROM employees;
```

SAMPLE RESULT

total_salary
265000

03

AVG()

Returns the average value of a numeric column.

EXAMPLE

```
SELECT AVG(salary) AS avg_salary
FROM employees;
```

SAMPLE RESULT

avg_salary
66250.00

04

MIN() & MAX()

Returns the minimum and maximum value in a column.

EXAMPLE

```
SELECT MIN(salary) AS min_salary,
MAX(salary) AS max_salary
FROM employees;
```

SAMPLE RESULT

min_salary	max_salary
50000	82000

**PRO TIP**

- Aggregate functions ignore NULL values.
- Use WHERE clause to filter data before aggregation for accurate results.

**INTERVIEW QUESTION**

What is the difference between COUNT(*) and COUNT(column)?

EXPECTED ANSWER:

- COUNT(*) counts all rows, including those with NULLs.
- COUNT(column) counts only non-NULL values in that column.



6

GROUP BY & HAVING



GROUP BY groups rows that have the same values.

HAVING filters the grouped results.



WHAT IT DOES?

Used to group data and apply conditions on groups.



EXAMPLE

```
SELECT department, COUNT(*) AS total_emp,
AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 60000;
```



WHY USE IT?

To summarize data like total sales per department.



SAMPLE TABLE: employees

id	name	department	salary
1	Riya	HR	60000
2	Aman	IT	75000
3	Karan	Finance	50000
4	Neha	IT	82000
5	Ravi	Finance	65000



PRO TIP

WHERE filters rows before grouping.

HAVING filters groups after grouping.



RESULT

department	total_emp	avg_salary
IT	2	78500.00
Finance	2	57500.00



INTERVIEW QUESTION

Can we use WHERE with aggregate functions?

Expected Answer: No, WHERE is used before aggregation.

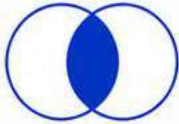
Use HAVING when you need to filter based on aggregate results.



7 JOINS



JOINS are used to combine rows from two or more tables based on a related column between them.

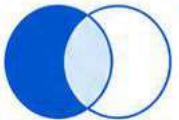


01 INNER JOIN

Returns only the matching rows from both tables.

Example

```
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id;
```



02 LEFT JOIN

Returns all rows from the left table and matched rows from the right table.

Example

```
SELECT e.name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id;
```



03 RIGHT JOIN

Returns all rows from the right table and matched rows from the left table.

Example

```
SELECT e.name, d.department_name
FROM employees e
RIGHT JOIN departments d
ON e.department_id = d.department_id;
```

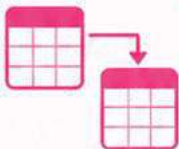


04 FULL JOIN

Returns all rows when there is a match in either left or right table.

Example

```
SELECT e.name, d.department_name
FROM employees e
FULL JOIN departments d
ON e.department_id = d.department_id;
```



05 SELF JOIN

Joins a table with itself.

Example

```
SELECT e1.name AS employee, e2.name AS manager
FROM employees e1
JOIN employees e2
ON e1.manager_id = e2.id;
```



PRO TIP

- Use table aliases (e, d) to make queries shorter and readable.
- SELF JOIN is useful for hierarchical data like employee-manager relationships.



INTERVIEW QUESTION

What is the difference between INNER JOIN and LEFT JOIN?

Expected Answer:

INNER JOIN returns only matching rows from both tables.

LEFT JOIN returns all rows from the left table, including non-matches.



8

SUBQUERIES



A **subquery** is a query nested inside another query.

It can be used in **WHERE**, **FROM**, or **SELECT** clause.

TYPES OF SUBQUERIES

- 1 Single Row Subquery
- 2 Multiple Row Subquery
- 3 Correlated Subquery
- 4 EXISTS / NOT EXISTS

01

SINGLE ROW SUBQUERY



Returns a single value.

Example

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

02

MULTIPLE ROW SUBQUERY

Returns multiple values.

Example

```
SELECT name, department_id
FROM employees
WHERE department_id IN
  (SELECT department_id
   FROM departments);
```



03

CORRELATED SUBQUERY



Subquery depends on outer query.

Example

```
SELECT e1.name, e1.salary
FROM employees e1
WHERE e1.salary > (SELECT AVG(e2.salary)
                  FROM employees e2
                  WHERE e2.department_id = e1.department_id);
```

04

EXISTS / NOT EXISTS

Checks existence of records.

Example

```
SELECT name
FROM employees e
WHERE EXISTS (SELECT 1 FROM departments d
              WHERE d.department_id = e.department_id);
```



PRO TIP

EXISTS is often faster than **IN** with large datasets.



INTERVIEW QUESTION

When should we use **EXISTS** instead of **IN**?

Expected Answer:

Use **EXISTS** when the subquery returns a large result set or when you only need to check the existence of records.



9

WINDOW FUNCTIONS



Window functions perform calculations across a set of table rows that are somehow related to the current row.

01

ROW_NUMBER()

#

Assigns a unique number to each row within its partition.

Example

```
SELECT name, salary,
ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn
FROM employees;
```

Sample Output

name	salary	rn
Riya	60000	1
Neha	52000	2
Aman	46000	3

02

RANK()



Assigns a rank with gaps when there are ties.

Example

```
SELECT name, salary,
RANK() OVER (ORDER BY salary DESC) AS rnk
FROM employees;
```

Sample Output

name	salary	rnk
Riya	60000	1
Neha	52000	2
Aman	52000	2
Karan	45000	4

03

DENSE_RANK()



Assigns a rank without gaps when there are ties.

Example

```
SELECT name, salary,
DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk
FROM employees;
```

Sample Output

name	salary	drnk
Riya	60000	1
Neha	52000	2
Aman	52000	2
Karan	45000	3

04

LAG()



Access data from the previous row.

Example

```
SELECT name, salary,
LAG(salary) OVER (ORDER BY salary) AS
prev_salary
FROM employees;
```

Sample Output

name	salary	prev_salary
Aman	46000	NULL
Karan	52000	46000
Neha	52000	52000
Riya	60000	52000

05

LEAD()



Access data from the next row.

Example

```
SELECT name, salary,
LEAD(salary) OVER (ORDER BY salary) AS
next_salary
FROM employees;
```

Sample Output

name	salary	next_salary
Aman	46000	52000
Karan	52000	52000
Neha	52000	60000
Riya	60000	NULL

★ PRO TIP

- Window functions do not collapse rows like GROUP BY.
- They return results without losing row level detail.



INTERVIEW QUESTION

What is the difference between RANK() and DENSE_RANK()?

Expected Answer:

RANK() leaves gaps in rank values when there are ties, while DENSE_RANK() does not.





CTE is a temporary result set that you can reference within a **SELECT**, **INSERT**, **UPDATE**, or **DELETE** statement.

01 WITH (BASIC CTE)

Makes a complex query easier to read and maintain.

```
WITH high_salary AS (
  SELECT name, salary
  FROM employees
  WHERE salary > 60000
)
SELECT * FROM high_salary;
```

Sample Table: employees

id	name	salary
1	Riya	60000
2	Aman	75000
3	Karan	50000
4	Neha	82000
5	Ravi	65000

✓ Result

name	salary
Aman	75000
Neha	82000
Ravi	65000

02 RECURSIVE CTE

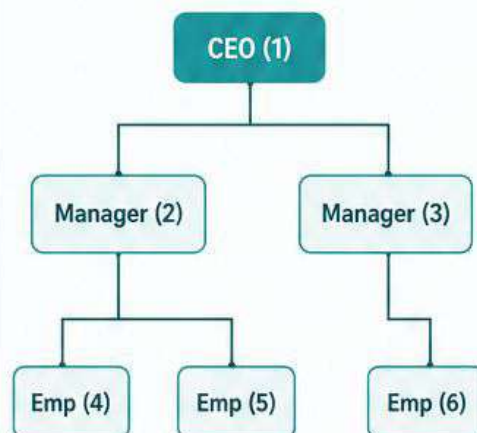
Used to work with hierarchical or tree-like data.

```
WITH RECURSIVE emp_hierarchy AS (
  -- Anchor member
  SELECT id, name, manager_id, 1 AS level
  FROM employees
  WHERE manager_id IS NULL

  UNION ALL

  -- Recursive member
  SELECT e.id, e.name, e.manager_id, eh.level + 1
  FROM employees e
  INNER JOIN emp_hierarchy eh
    ON e.manager_id = eh.id
)
SELECT * FROM emp_hierarchy;
```

Employee Hierarchy



Pro Tips

- ✓ Use CTEs to break down complex queries into simpler parts.
- ✓ Recursive CTEs are perfect for hierarchical and tree structures.
- ✓ CTEs exist only for the duration of the query execution.



Interview Question



When should you use a CTE instead of a subquery?

Expected Answer:

CTEs improve readability, allow reusability, and are easier to debug—especially for complex queries.